Packaging HTTP Live Streaming (HLS) with TS

Table of Contents

- Options for HLS packaging with Transport Streams (HLS TS)
- Protocol version
- Using multiplexed audio/video (protocol version 3)
- Using multiplexed audio/video with AES-128 (protocol version 3)
- Using late binding of audio/video groups (protocol version 4)
- Signaling embedded 608/708 Closed Captions (protocol version 4)
- Using late binding of audio/video groups with SAMPLE-AES (protocol version 5)
- Using late binding of audio/video groups with SAMPLE-AES for Apple FairPlay
- Audio only Sample AES
- Adding WebVTT subtitles when packaging HLS TS
- Overriding default playlist values
- Advanced recipes

`mp4split` generates all files required for HLS streaming using MP4 audio/video files as input. To enable the HLS packaging mode with Transport Streams (TS), the first option on the command-line must be `--package-hls`.

> **Note**
>
> Unified Packager automatically sets the correct HLS protocol version for the Master and Media Playlists, based on the functionality used in each playlist. Please note that it is valid for a Media Playlist to signal a protocol version that is higher than the protocol version signaled in the Master Playlist. Please also note that the HLS protocol version that introduced the latest functionality that is used in a playlist is not necessarily the protocol version that will be signaled, as signaling a specific protocol version is only required for functionality that breaks backwards compatibility: 'Protocol Version Compatibility' paragraph of the HLS specification.

The following is a list of all options specific for HLS packaging.

Options for HLS packaging with Transport Streams (HLS TS)

--package_hls

Enable the HLS packaging mode with Transport Streams.

--fragment_duration

Specifies the duration of each fragment as a fraction of seconds. It is useful to choose the denominator to reflect framerate so that the numerator naturally equals (an integer multiple of) the GOP size (e.g. `192/24` rather than `8/1`). For legacy reasons, the value is taken to reflect **milliseconds** when the denominator is omitted (e.g. `--fragment_duration=8000`). When combined with `--start_segments_with_iframe` Packager will create fragments that contain one or more full GOPs and not exceed the specified duration (i.e., it will regard the value as a maximum).

--output_single_file

Output the fragments as a single file, instead of a separate file for each fragment.

--base_media_file

The base name of the media file (defaults to `fileSequence`).

--encrypt_key_file

The file that holds the key information.

--encrypt_key_url

The license acquisition absolute URL.

--encrypt_iv

The 128 bit AES Initialization Vector (IV). This is a random 128 bit value. The default is to use the sequence number.

--stream_encrypt

Use SAMPLE-AES for encryption instead of AES-128.

--streaming_key_delivery

Use com.apple.streamingkeydelivery as KEYFORMAT.

--start_segments_with_iframe

When enabled, Packager will make sure that each fragment starts with an IDR frame and create fragments that contain one or more full GOPs. By default, this option is disabled.

--iframe_index_file

The name of the iframe index file (defaults to `iframe_index.m3u8`).

Protocol version

The protocol version depends on the features you are requesting, so you don't have to set this explicitly. E.g. enabling output-single-file requires a minimum protocol version of 4 because `EXT-X-BYTERANGE` tags are used.

Newer versions of the protocol supports dynamic binding of audio, video and subtitles. Older versions require multiplexed audio/video. Let's start by creating content that is using multiplexed audio/video and gives the highest degree of compatibility with devices/players.

Using multiplexed audio/video (protocol version 3)

The following example creates two MP4's that each contain audio and video, with the bitrate for the video being different for each. Then, those MP4's are used as input files for packaging multiplexed HLS.

```
#!/bin/bash

mp4split -o presentation-1.mp4 \
  tears-of-steel-avc1-1000k.mp4 \
  tears-of-steel-aac-128k.mp4
mp4split -o presentation-2.mp4 \
  tears-of-steel-avc1-400k.mp4 \
  tears-of-steel-aac-128k.mp4
```

The command-lines below will generate a directory each that contains the Media Playlists and the `.ts`-files that are created from the MP4's:

```bash
#!/bin/bash

mp4split \
  --package_hls \
  -o presentation_s1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=s1_ \
  presentation-1.mp4

mp4split \
  --package_hls \
  -o presentation_s2/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=s2_ \
  presentation-2.mp4
```

Output files for multiplexed audio/video

Filename	Description
presentation_s1/prog_index.m3u8	The playlist for the 1st stream.
presentation_s1/s1_NNN.ts	A number of MPEG TS segments for the 1st stream.
presentation_s2/prog_index.m3u8	The playlist for the 2nd stream.
presentation_s2/s2_NNN.ts	A number of MPEG TS segments for the 2nd stream.

Then the two Media Playlists are used as input two create a Master Playlist:

```bash
#!/bin/bash

mp4split --package_hls -o presentation.m3u8 \
  presentation_s1/prog_index.m3u8 \
  presentation_s2/prog_index.m3u8
```

Using multiplexed audio/video with AES-128 (protocol version 3)

Now let us add AES-128 encryption to the previous example. First we have to create a 128-bit CEK (Content Encryption Key). This is just a 16 bytes file with random bytes. You could use for example 'openssl' to create the key:

```
#!/bin/bash

openssl rand 16 > presentation.key
```

The command-lines for segmenting the mp4 files is the same as above, except that we need to add two additional options.

We have to pass the presentation.key as the `--encrypt-key-file` option.

Since the player needs to know where to fetch the encryption key we pass the same presentation.key also to the `--encrypt-key-url` option.

It's also possible to specify a full HTTP(s) URL or your own URL format.

The corresponding command lines are:

```
#!/bin/bash

# URL that resolves to the presentation.key file
LA_URL=https://license-server/presentation.key

mp4split --package_hls -o presentation_s1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=s1_ \
  --encrypt_key_file=presentation.key \
  --encrypt_key_url=${LA_URL} \
  presentation-1.mp4
```

and

```
#!/bin/bash

# URL that resolves to the presentation.key file
LA_URL=https://license-server/presentation.key

mp4split --package_hls -o presentation_s2/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=s2_ \
  --encrypt_key_file=presentation.key \
  --encrypt_key_url=${LA_URL} \
  presentation-2.mp4
```

The creation of the variant playlist is the same as above:

```
#!/bin/bash

mp4split --package_hls -o presentation.m3u8 \
  presentation_s1/prog_index.m3u8 \
  presentation_s2/prog_index.m3u8
```

You will now have the following files and directories for this presentation:

Output files for multiplexed audio/video

Filename	Description
presentation.m3u8	The variant/master playlist.
presentation.key	The 16 bytes Content Encryption Key.
presentation_s1/prog_index.m3u8	The playlist for the 1st stream.
presentation_s1/s1_NNN.ts	A number of MPEG TS segments for the 1st stream.
presentation_s2/prog_index.m3u8	The playlist for the 2nd stream.
presentation_s2/s2_NNN.ts	A number of MPEG TS segments for the 2nd stream.

Download

Please download the package-hls-aes-v3.sh sample script which creates the various server manifest as discussed above. The sample content is Tears of Steel.

Using late binding of audio/video groups (protocol version 4)

If you have more than one audio track (e.g. different codecs, different languages) and/or subtitles then you want to use the grouping that the HLS protocol supports. In this case all the audio/video/subtitles have to be segmented separately.

Input files for grouped audio/video

Filename	Description
presentation-1.mp4	AVC 416x234 @ 600kbps, AAC-SBR @ 64kbps
presentation-2.mp4	AVC 416x234 @ 800kbps, AAC-LC @ 128 kbps
presentation-3.mp4	AVC 640x360 @ 1200 kbps, AC3 @ 128 kbps

Filename	Description
image.png	PNG image to embed in AAC-SBR stream

The mp4split commands are:

```
#!/bin/bash

mp4split --package_hls -o presentation_v1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=v1_ \
  tears-of-steel-avc1-1500k.mp4

mp4split --package_hls -o presentation_v2/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=v2_ \
  tears-of-steel-avc1-1000k.mp4

mp4split --package_hls -o presentation_v3/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=v3_ \
  tears-of-steel-avc1-400k.mp4

mp4split --package_hls -o presentation_a1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=a1_ \
  tears-of-steel-ac3-448k.mp4

mp4split --package_hls -o presentation_a2/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=a2_ \
  tears-of-steel-aac-128k.mp4

mp4split --package_hls -o presentation_a3/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --base_media_file=a3_ \
  tears-of-steel-aac-64k.mp4
```

The following output is created:

Output files for multiplexed audio/video

Filename	Description
presentation_v1/prog_index.m3u8	The playlist for the 1st video stream.
presentation_v1/iframe_index.m3u8	The IFRAME only playlist for the 1st video stream.
presentation_v1/v1_NNN.ts	A number of MPEG TS segments for the 1st video stream.
presentation_v2/prog_index.m3u8	The playlist for the 2nd video stream.

Filename	Description
presentation_v2/iframe_index.m3u8	The IFRAME only playlist for the 2nd video stream.
presentation_v2/v2_NNN.ts	A number of MPEG TS segments for the 2nd video stream.
presentation_v3/prog_index.m3u8	The playlist for the 3rd video stream.
presentation_v3/iframe_index.m3u8	The IFRAME only playlist for the 3rd video stream.
presentation_v3/v3_NNN.ts	A number of MPEG TS segments for the 3rd video stream.
presentation_a1/prog_index.m3u8	The playlist for the 1st audio stream.
presentation_a1/a1_NNN.aac	A number of AAC segments for the 1st audio stream (including
presentation_a2/prog_index.m3u8	The playlist for the 2nd audio stream.
presentation_a2/a2_NNN.aac	A number of AAC segments for the 2nd audio stream.
presentation_a3/prog_index.m3u8	The playlist for the 3rd audio stream.
presentation_a3/a3_NNN.ac3	A number of AC3 segments for the 3rd audio stream.

Since we use a higher protocol version we can also add the iframe_index.m3u8 files to the Master Playlist.

```
#!/bin/bash

mp4split --package_hls -o presentation.m3u8 \
  presentation_v1/prog_index.m3u8 \
  presentation_v1/iframe_index.m3u8 \
  presentation_v2/prog_index.m3u8 \
  presentation_v2/iframe_index.m3u8 \
  presentation_v3/prog_index.m3u8 \
  presentation_v3/iframe_index.m3u8 \
  presentation_a1/prog_index.m3u8 \
  presentation_a2/prog_index.m3u8 \
  presentation_a3/prog_index.m3u8
```

If you want to add an audio-only variant to the variant playlist, you can simply list the audio playlist twice on the command-line. And when you work with multiple languages, you should do this for every language, which will result in an audio-only variant that references an audio group that contains the audio for each of the languages. Do note that for each additional audio playlist in a different language that you add to the command-line twice, `mp4split` will return a message saying that the group has already been added for audio-only. This is an informational message that can be ignored.

Download

Please download the package-hls-aes-v4.sh sample script which creates the various server manifest as discussed above. The sample content is Tears of Steel.

Signaling embedded 608/708 Closed Captions (protocol version 4)

When the video track contains embedded closed captions they need to be signaled separately. The default is to signal that the video tracks do not contain any closed captions, so if they do contain captions you have to create a Closed Captions playlist and explicitly add it to the variant playlist.

The input file is the video presentation that contains the embedded Closed Captions. This time we specifically select the text track and at the same time assign it a description.

The mp4split command is:

```
#!/bin/bash

mp4split --package_hls -o presentation_c1/prog_index.m3u8 \
  example-with-embedded-captions.mp4 \
    --track_type=text \
    --track_description="Closed Captions" \
    --track_language="eng"
```

This creates a 'presentation_c1/prog_index.m3u8' file which can then be added to the Master Playlist (identical to the previous example).

> **Note**
>
> Embedded captions are not extracted, just signaled.

Using late binding of audio/video groups with SAMPLE-AES (protocol version 5)

Now let us add SAMPLE-AES encryption to the previous example. First we have to create a 128-bit CEK (Content Encryption Key). This is just a 16 bytes file with random bytes. You could use for example 'openssl' to create the key:

```
#!/bin/bash

openssl rand 16 > presentation.key
```

The command-lines for segmenting the MP4 files is the same as above, except that we need to add four additional options.

We have to pass the presentation.key as the `--encrypt-key-file` option.

Since the player needs to know where to fetch the encryption key we pass the same presentation.key also to the `--encrypt-key-url` option.

It's also possible to specify a full HTTP(s) URL or your own URL format.

We also specify `--stream-encrypt` to enable SAMPLE-AES encryption

The mp4split command is:

```
#!/bin/bash

# URL that resolves to the presentation.key file
LA_URL=https://license-server/presentation.key

mp4split --package_hls -o presentation_v1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --encrypt_key_file=presentation.key \
  --encrypt_key_url=${LA_URL} \
  --stream_encrypt \
  --base_media_file=v1_ \
  tears-of-steel-avc1-1500k.mp4
```

Repeat this for presentation_v2, presentation_v3, presentation_a1, presentation_a2, presentation_a3.

Creating the Master Playlist is identical to the previous example.

Using late binding of audio/video groups with SAMPLE-AES for Apple FairPlay

For FairPlay we first have to create a 128-bit CEK (Content Encryption Key) and 128-bit IV (Initialization Vector). This is just a 32 bytes file with random bytes, where the first 16 bytes represent the CEK and the remaining 16 bytes represent the key IV. You could use for example

'openssl' to create the key.

```
#!/bin/bash

openssl rand 32 > presentation.key
```

The command-lines for segmenting the mp4 files is the same as above and we also specify `--stream-encrypt` to enable SAMPLE-AES encryption and `--streaming-key-delivery` for FairPlay.

The mp4split command is:

```
#!/bin/bash

# URL that resolves to the presentation.key file
LA_URL=https://license-server/presentation.key

mp4split --package_hls -o presentation_v1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --encrypt_key_file=presentation.key \
  --encrypt_key_url=${LA_URL} \
  --stream_encrypt \
  --streaming_key_delivery \
  --base_media_file=v1_ \
  tears-of-steel-avc1-1500k.mp4
```

Repeat this for presentation_v2, presentation_v3, presentation_a1, presentation_a2, presentation_a3.

Creating the Master Playlist is identical to the previous example.

Please note that within FairPlay the encrypt-key-url is only for information, a client typically implements different means of locating the license server and fetch keys.

Audio only Sample AES

Using the packager it is possible to create a Sample AES protected audio only stream that includes a 'still image' (a jpg or png shown with the audio stream).

The image is embedded (with an ID3 APIC tag) in the AAC audio.

The example command is the following:

```bash
#!/bin/bash

# URL that resolves to the video.key file
LA_URL=https://license-server/video.key

mp4split --package_hls -o prog_index.m3u8 \
  --fragment_duration=192/24 \
  --encrypt_key_file=video.key \
  --stream_encrypt \
  --encrypt_key_url=${LA_URL} \
  --output_single_file \
  tears-of-steel-aac-64k.mp4 \
  hls.jpg
```

This creates a .m3u8 that looks like the following:

```
#EXTM3U
#EXT-X-VERSION:5
## Created with Unified Streaming Platform(version=1.6.8)
#EXT-X-MEDIA-SEQUENCE:0
#EXT-X-PLAYLIST-TYPE:VOD
#EXT-X-TARGETDURATION:10
#EXT-X-KEY:METHOD=SAMPLE-AES,URI="video.key"
#EXTINF:9.002, no desc
#EXT-X-BYTERANGE:40775@0
fileSequence.aac

...

#EXTINF:1.088, no desc
#EXT-X-BYTERANGE:15902@1095409
fileSequence.aac
#EXT-X-ENDLIST
#USP-X-MEDIA:TYPE=AUDIO,GROUP-ID="audio-aacl-
65",LANGUAGE="en",NAME="English",AUTOSELECT=YES,BANDWIDTH=117000,CODECS="mp4a.40.2"
```

Note

The filename 'fileSequence.aac' is the default filename, which can be change by the use of '--base-media-file'.

Safari (or Quicktime or Apple TV) has no support for the ID3 tag used, so it does play (the sound) but no image is shown.

Dolby Digital Plus, also known as Enhanced AC-3 or EC-3 audio streams are supported as well. See here for further reference HLS Sample-AES Audio Formats.

Adding WebVTT subtitles when packaging HLS TS

To add WebVTT subtitles when packaging HLS TS, do the following:

1. For each subtitles track that you want to add, make sure that you have packaged the WebVTT, SRT or TTML formatted source files in fMP4 (`.ismt` or `.cmft`), as explained in Packaging Subtitles.
2. For each fMP4 that contains a subtitles track that you want to add to your stream, use `mp4split` with the `--package_hls`-option to create a Media Playlist and generate the WebVTT Segments at the same time (note that the output will be WebVTT, regardless of whether the input is formatted as TTML or as WebVTT). At this stage, you can also override the automatically generated track description for HLS's 'NAME' attribute by using `--track_description`, and override the default length of the WebVTT Segments using `--fragment_duration`:

```
#!/bin/bash

mp4split --package_hls -o example_c1/prog_index.m3u8 \
  --fragment_duration=60/1 \
  tears-of-steel-fr.ismt \
  --track_description="Sous-titres Français"
```

3. When you are ready to create the Master Playlist, add the relevant Media Playlists for subtitles to your input for `mp4split` like you add all other Media Playlists:

```
#!/bin/bash

mp4split --package_hls -o example.m3u8 \
  example_c1/prog_index.m3u8 \
  example_v1/prog_index.m3u8 \
  example_v1/iframe_index.m3u8

  ... (other elements of the presentation)
```

Overriding default playlist values

`mp4split` uses some default rules when generating the NAME and GROUP-ID fields.

If you are not happy with the defaults, you can specify the following options on the command line after each input file.

mp4split options for overriding default values

Option	Description
track_groupid	Specify the value used for the GROUP-ID field in the EXT-X-MEDIA tag.
track_description	Specify the value used for the NAME field in the EXT-X-MEDIA tag.

An example command-line that shows how the `--track_groupid` and `--track_description` options can be used, is shown below. Note that the grouping defined in this command-line is identical to the default behaviour:

```
#!/bin/bash

mp4split --package_hls -o \
  audio_eng.isma --track_groupid=aac --track_description="English" \
  audio_deu.isma --track_groupid=aac --track_description="Deutsch" \
  commentary.isma --track_groupid=aac \
  --track_description="Commentary" --track_name="audio_commentary" \
  video_128k.ismv video_256k.ismv video_768k.ismv
```

Warning

Keep in mind that using the `--track_groupid` option will influence the variant streams that are generated and that using this option can therefore easily create a stream that does not follow Apple's recommendations and break playout. This is why we strongly advise against configuring HLS media groups yourself.

Advanced recipes

Using PlayReady Envelope encryption for HLS

New in version 1.7.14.

You can encrypt the media segments using PlayReady Envelope encryption.

This encryption mode is set by specifying the option `--hls.playout=playready_envelope`. The PlayReady key information option (`--iss.key` and `--iss.license_server_url`) are described at Adding PlayReady Envelope DRM.

Example

```bash
#!/bin/bash

  KID=10000000100010001000100000000001
  CEK=3a2a1b68dd2bd9b2eeb25e84c4776668
  KID_UUID=10000000-1000-1000-1000-100000000001 #UUID representation of KID
  CEK_B64="OiobaN0r2bLusl6ExHdmaA==" #Base64 byte array representation of CEK
  LA_URL="https://test.playready.microsoft.com/service/rightsmanager.asmx?cfg=
(kid:$KID_UUID,contentkey:$CEK_B64,ckt:aesctr)"

  # first the media and the media playlist
  mp4split --package_hls \
    -o presentation_s1/prog_index.m3u8 \
    --fragment-duration=192/24 \
    --output-single-file \
    --hls.playout=playready_envelope \
    --iss.license_server_url=${LA_URL} \
    --iss.key=${KID}:${CEK} \
    tears-of-steel-avc1-1000k.mp4

  # then the master playlist
  mp4split --package_hls \
    -o presentation.m3u8 \
    presentation_s1/prog_index.m3u8
```

Using Discretix PlayReady encryption for HLS

New in version 1.7.19.

You can encrypt the media segments using Discretix PlayReady encryption.

This encryption mode is set by specifying the option `--hls.playout=fixture`. The PlayReady key information option (`--tuxedo.key` and `--text.license_server_url`) are described at Adding Viaccess-Orca (Discretix) PlayReady for HLS.

Example

```bash
#!/bin/bash

KID=the-key-id
CEK=the-content-key
LA_URL=the-license-server-url

# first the media and the media playlist
mp4split --package_hls \
  -o presentation_s1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --output_single_file \
  --hls.playout=dxdrm \
  --dxdrm.license_server_url=${LA_URL} \
  --dxdrm.key=${KID}:${CEK} \
  tears-of-steel-avc1-1000k.mp4

# then the master playlist
mp4split --package_hls \
  -o presentation.m3u8 \
  presentation_s1/prog_index.m3u8
```

Adobe Primetime DRM

HLS can also be packaged with Adobe Primetime DRM SAMPLE-AES.

By adding `--hds.drm_specific_data` the Adobe Primetime DRM signaling and DRM data will be present in the media playlist .m3u8.

Example

```bash
KEYFILE=/path/to/sample-aes.key
IV=00000000000000000000000000000001
URL=url://to/key/acquisition
DRM_SPECIFIC_DATA="TG9yZW0gaXBzdW0gZG9sb3Igc2l0IGFtZXQsIGNvbnNlY3RldHbnNlY3Rl..."

# first the media and the media playlist
mp4split --package_hls \
  -o presentation_s1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --encrypt_key_file=${KEYFILE} \
  --encrypt_key_url=${URL} \
  --stream_encrypt \
  --encrypt_iv=$(IV) \
  --hds.drm_specific_data=${DRM_SPECIFIC_DATA} \
  tears-of-steel-avc1-1000k.mp4

# then the master playlist
mp4split --package_hls \
  -o presentation.m3u8 \
  presentation_s1/prog_index.m3u8
```

China DRM

New in version 1.7.19.

Using the `--key-format="chinadrm"` option, the content will be packaged with China DRM, and the signaling will be present in the media playlists.

Example

```
KEYFILE=/path/to/content_encryption.key
URL=url://to/key/acquisition
IV=key_iv_in_hex

# first the media and the media playlist
mp4split --package_hls \
  -o presentation_s1/prog_index.m3u8 \
  --fragment_duration=192/24 \
  --encrypt_key_file=${KEYFILE} \
  --encrypt_key_url=${URL} \
  --encrypt_iv=${IV} \
  --key_format="chinadrm"
  tears-of-steel-avc1-1000k.mp4

# then the master playlist
mp4split --package_hls \
  -o presentation.m3u8 \
  presentation_s1/prog_index.m3u8
```

Using CPIX to specify signaling for DRM system of choice

By specifying a CPIX document and Using explicitly specified signaling for '<HLSSignalingData>', it is possible to specify DRM signaling for any DRM system of choice. One example that is only supported using this method is Widevine for HLS TS. Speciying a CPIX document when statically packaging HLS TS is done as shown below (for a more advanced example using fMP4 HLS, see Multi-DRM protected HLS and DASH from a shared CMAF source):

```bash
#!/bin/bash

mp4split --package-hls \
  -o tos-1000k.m3u8 \
  --output-single-file \
  --cpix=example.cpix \
  --base-media-file=v1 \
  tears-of-steel-avc1-1000k.mp4

mp4split --package-hls \
  -o tos-128k.m3u8 \
  --output-single-file \
  --cpix=example.cpix \
  --base-media-file=a1 \
  tears-of-steel-aac-128k.mp4

mp4split --package-hls \
  -o tos.m3u8 \
  --output-single-file \
  --cpix=example.cpix \
  tos-1000k.m3u8 \
  tos-128k.m3u8
```